

SUMÁRIO

1 CONTEXTUALIZAÇÃO	1
2 DESAFIO	2
3 RESULTADOS ESPERADOS (ENTREGA)	2
4 REQUISITOS DA APLICAÇÃO	3
5 ROTEIRO DA APLICAÇÃO	5
5.1 FORMATO DO SISTEMA	5
5.2 DOCUMENTAÇÃO NO README.MD	5
5.3 USO DO GITHUB	6
5.4 GRAVAÇÃO DE VÍDEO	6
6 CRITÉRIOS DE AVALIAÇÃO	7
7 PLANO DE PROJETO	8
8 SOBRE O USO DE INTELIGÊNCIA ARTIFICIAL	8

1 CONTEXTUALIZAÇÃO

O desenvolvimento de modelos de Inteligência Artificial exige a execução das etapas de preparação de dados (Data Prep), engenharia de recursos (Feature Engineering) e análise exploratória. O treinamento de algoritmos com dados inconsistentes ou desbalanceados causa o fenômeno Garbage In, Garbage Out (Lixo Entra, Lixo Sai), gerando modelos com sobreajuste (overfitting) que apresentam alto desempenho na fase de treino, mas reduzem a acurácia quando expostos a novos dados.

Neste projeto de encerramento de módulo, você aplicará o fluxo de trabalho de um Cientista de Dados: executará o tratamento estatístico de uma base histórica, construirá

uma nova variável numérica, controlará o overfitting por meio do ajuste de hiperparâmetros e selecionará o modelo classificador final com base na métrica de acurácia.

2 DESAFIO

Você atuará como Cientista de Dados para estruturar um Pipeline Preditivo completo aplicado à Indústria 4.0.

- **O Problema:** Um parque fabril monitorado por sensores necessita prever quebras mecânicas nos equipamentos para evitar paradas na linha de produção. A variável alvo do projeto é binária: Falha = 1 (quando há uma avaria detectada) e Falha = 0 (funcionamento normal).
- **Link da Base de Dados (.csv):**
https://drive.google.com/drive/folders/1_QcYvhSoJO6SxOJz8Om6gaVuWYpZKdMs?usp=sharing
- *Além da base de dados, o engenheiro responsável disponibilizou alguns detalhes importantes que você precisa levar em consideração: anotações do Departamento de Engenharia.*

3 RESULTADOS ESPERADOS (ENTREGA)

O código deverá ser inserido e versionado no **GitHub em modo público**, o vídeo deverá ser inserido no **Google Drive em modo leitor para qualquer pessoa** com o link. Ambos os **links** deverão ser disponibilizados na tarefa **Módulo 1 - Projeto Avaliativo**, presente na **semana 13** do AVA até o dia **20/07/2026 às 22h**.

Não serão aceitos projetos submetidos após a data limite da atividade, e, ou alterados depois de entregues.

Importante:

1. Será considerado como data final de entrega a última atualização no repositório do projeto no GitHub. Lembre-se de não modificar o código até receber sua nota.
2. Não esqueça de submeter os links no AVA. Os links não submetidos terão penalidade na nota.

4 REQUISITOS DA APLICAÇÃO

O desenvolvimento técnico do seu código Python no Notebook deve, obrigatoriamente, ser segmentado e documentado nas seguintes etapas estruturadas (pode-se utilizar essa mesma estrutura em um projeto .py com separação em módulos):

Fase 1: Análise Exploratória (EDA)

- Apresente as dimensões do dataset (número de linhas e colunas), os tipos de dados das variáveis e o resumo estatístico descritivo das colunas numéricas via método “.describe()”.
- Plote, no mínimo, 3 gráficos analíticos bem fundamentados utilizando bibliotecas como Matplotlib ou Seaborn (Ex: histograma de distribuição das variáveis preditoras, gráfico de barras comprovando a taxa de desbalanceamento da variável alvo e um mapa de calor com a correlação de Pearson entre as variáveis).
- Insira uma célula de texto analisando os valores numéricos e os padrões identificados nos gráficos, explicitando como eles direcionam a estratégia de modelagem.

Fase 2: Limpeza e Tratamento de Dados (Data Prep)

- Identifique e remova as linhas duplicadas.
- Identifique dados ausentes e aplique a imputação por Média ou Mediana, justificando textualmente a escolha com base na distribuição dos dados.
- Gere gráficos do tipo *boxplot* para identificar a presença de outliers nas variáveis explicativas.

Fase 3: Feature Engineering

- Crie uma nova coluna numérica por meio de operação matemática entre colunas existentes, tratando os valores nulos previamente.
- *Sugestão (Manutenção): $potencia = velocidade_rotacao_rpm * torque_nm$*
- *(É permitida a criação de outra combinação matemática, desde que explicada no vídeo e documentada no notebook).*

Fase 4: Divisão e Balanceamento dos Dados

- Separe as variáveis preditoras (X) da variável alvo (y).
- Divida os dados em treino (80%) e teste (20%) utilizando o parâmetro `stratify=y`.
- Aplique uma técnica de reamostragem (SMOTE ou Random Under Sampling) exclusivamente nos dados de treino para evitar o vazamento de dados (*Data Leakage*).

Fase 5: Escalonamento de Variáveis (StandardScaler)

- Aplique o StandardScaler apenas nas variáveis contínuas destinadas ao modelo KNN (utilizando `fit_transform` no treino e `transform` no teste).
- Mantenha os dados da Árvore de Decisão sem escalonamento, justificando no código o motivo de o algoritmo ser imune à escala dos atributos.

Fase 6: Ajuste de Parâmetros e Combate ao Overfitting

- No **KNN**: Treine o modelo variando o parâmetro `n_neighbors` (K) por no mínimo 3 valores ímpares (ex: K = 3, 5, 7) e registre a acurácia no treino e no teste.

- Na **Árvore**: Treine o modelo variando o parâmetro `max_depth` por no mínimo 3 limites (ex: 3, 5 e None) e registre a acurácia no treino e no teste.
- Insira um texto identificando em quais pontos ocorreu o *overfitting* e qual configuração garantiu a estabilidade no teste.

Fase 7: Avaliação da Acurácia e Veredito Final

- Calcule e exiba a acurácia final do melhor KNN e da melhor árvore de decisão utilizando os dados de teste.
- Compare as taxas de acerto e escreva uma conclusão justificando qual modelo apresentou o desempenho superior no teste e deve ser adotado pela empresa.

5 ROTEIRO DA APLICAÇÃO

Abaixo segue o detalhamento de como a solução de Machine Learning deve ser estruturada, organizada e entregue no repositório do projeto.

5.1 FORMATO DO SISTEMA

A aplicação é um Pipeline de Ciência de Dados ponta a ponta construído em Python. O repositório do projeto deve seguir a estrutura e organização de entrega:

- **Notebook Principal (.ipynb)**: O código completo da solução deve ser entregue em um único arquivo de Notebook Jupyter. Ele precisa ser dividido explicitamente em blocos (cabeçalhos de Markdown) que separem de forma organizada as 6 fases descritas nos *Requisitos da Aplicação*.
- **Código Limpo e Explicativo**: Todas as células de código que realizam transformações matemáticas, limpezas ou treinamentos devem vir acompanhadas de células de texto justificando a tomada de decisão técnica (ex: por que escolheu a média ou a mediana, por que podou a árvore naquele nível, etc.).
- **Reprodutibilidade (requirements.txt)**: O repositório deve conter um arquivo de texto listando todas as dependências de bibliotecas Python utilizadas (como pandas, scikit-learn, seaborn e suas respectivas versões), permitindo que qualquer outro cientista de dados consiga recriar o ambiente e rodar o pipeline sem erros.
- **Caminhos Relativos de Dados**: Se os dados forem baixados e lidos localmente no repositório (em vez de carregados por URL direta), eles devem ser organizados em uma pasta de dados específica (ex: /data). É proibido o uso de caminhos absolutos que apontem para diretórios locais da sua máquina (ex: C:/Usuarios/SeuNome/Documents/...), pois isso impede a execução do código pelo avaliador.

5.2 DOCUMENTAÇÃO NO README.MD

Crie um arquivo `readme.md` no repositório do seu projeto no GitHub, para documentar a sua solução, bem como demonstrar as técnicas e linguagens utilizadas, além do escopo do projeto e como o usuário pode executar o seu sistema.

Algumas dicas interessantes para utilizar na criação do seu portfólio são:

- Criar um nome para o seu software;
- Descrever qual o problema ele resolve;
- Descrever quais técnicas e tecnologias utilizadas. Aqui você também pode inserir alguma imagem ou diagrama para melhor entendimento;
- Descrever como executar;
- Descrever quais melhorias podem ser aplicadas;
- Entre outras coisas.

5.3 USO DO GITHUB

Utilizar um repositório GitHub para controle de versionamento do projeto. Siga as diretrizes de boas práticas de uso do Git abaixo:

- Seguir *commits* com descrição sucinta e direta e, de preferência, com poucos arquivos alterados por *commit*.
 - Prefira mensagens como “implementa X” ou “corrige Y”, em vez de “X foi implementado” ou “Y foi corrigido”.
- Crie uma *branch* “develop” para concentrar os merges enquanto estiver desenvolvendo as funcionalidades.
- Utilizar *feature branches* a partir da *develop* para cada tarefa.
 - Não excluir a branch após o pull request concluído.
- O código final do projeto deve estar todo “mergeado” no *branch* “main”.

5.4 GRAVAÇÃO DE VÍDEO

Além do desenvolvimento deste sistema você deverá gravar um vídeo, com tempo **máximo de 7 minutos**, abordando os seguintes questionamentos:

- Qual o objetivo do sistema? E demonstração de funcionamento.
- O que deve ser realizado para executar o sistema?
- Como você organizou as tarefas antes de começar a desenvolver?

- Quais as *branches* você criou e quais os objetivos para cada uma?
- Você acha que faltou algo no seu código que você poderia melhorar?
- Argumentação clara, baseada nos conteúdos abordados, de suas escolhas realizadas nesse projeto.

Você poderá gravar na vertical ou na horizontal. É importante que apareça seu rosto e esteja em um local com boa iluminação. Para realizar a entrega do vídeo, coloque em uma pasta do Google Drive em modo leitor para qualquer pessoa com o link, e compartilhe o mesmo na submissão do projeto no AVA. Uma dica interessante é você inserir o vídeo no `readme.md` do seu projeto no repositório do GitHub.

Note: Não devem ser utilizadas qualquer ferramenta de criação de vídeo com IA nessa etapa.

6 CRITÉRIOS DE AVALIAÇÃO

A tabela abaixo apresenta os critérios que serão avaliados durante a correção do projeto. O mesmo possui variação de nota de 0 (zero) a 10 (dez) como nota mínima e máxima, e possui peso de 60% sobre a avaliação do módulo.

Serão desconsiderados e atribuída a nota 0 (zero) os projetos que apresentarem plágio de soluções encontradas na internet ou de outros colegas. Lembre-se: Você está livre para utilizar outras soluções como base, mas não é permitida a cópia.

Apresentação do Projeto

Nº	Critério de Avaliação	0	2,0
1	Realizou a gravação de um vídeo?	Não foi realizada a gravação do vídeo.	Gravou o vídeo e abordou todos os tópicos listados no item 5.4

Uso adequado do GitHub e README.md

Nº	Critério de Avaliação	0	0,25	0.5
5	Foi aberto um branch separado para cada etapa, com mensagens	Os commits foram realizados diretamente na master/main, e	Foram abertas branches para cada funcionalidade, porém foram realizados commits de mesma	Foram abertas branches para cada funcionalidades, e os commits foram realizados de forma concisa com

	do commit claras e adequadas?	não foi usado mensagens claras.	funcionalidade em branchs diferentes, ou mensagens inadequadas.	mensagens adequadas.
6	Criou uma documentação com readme.md?	Não criou a documentação.	Criou uma documentação, porém de forma muito simplificada.	Criou uma documentação completa com ao menos todos os tópicos sugeridos no item 4.3

Desenvolvimento adequado da Aplicação

Nº	Critério de Avaliação	0	0.5	1.0
7	Análise Exploratória (EDA)	NÃO REALIZADO. Ausência de cálculo de dimensões, describe ou plotagem dos 3 gráficos solicitados.	PARCIALMENTE. Apresentou os dados e gráficos, mas omitiu a célula de texto interpretativa.	TOTALMENTE. Apresentou tamanho, método describe, os 3 gráficos solicitados e a análise em texto.
8	Limpeza e Tratamento de Dados (Data Prep)	NÃO REALIZADO. Ausência de remoção de duplicados, tratamento de nulos ou análise de outliers.	PARCIALMENTE. Realizou o tratamento dos dados, mas omitiu a justificativa da escolha entre média e mediana.	TOTALMENTE. Removeu duplicados, tratou nulos com justificativa técnica e realizou análise de outliers via boxplot.
9	Feature Engineering	NÃO REALIZADO. Não gerou nova coluna por meio de combinação matemática.	PARCIALMENTE. Criou a coluna, mas a operação resultou em erros de execução ou valores inválidos (NaN).	TOTALMENTE. Criou a nova coluna utilizando operação matemática validada e sem erros de execução.
10	Divisão e Balanceamento	NÃO REALIZADO. Não realizou o split treino/teste ou aplicou o balanceamento	PARCIALMENTE. Realizou a divisão 80/20, mas aplicou o balanceamento sem	TOTALMENTE. Dividiu a base em 80/20 com stratify e aplicou o balanceamento

		antes da divisão dos dados.	o parâmetro de estratificação.	exclusivamente nos dados de treino.
11	Escalonamento de Variáveis (StandardScaler)	NÃO REALIZADO. Não aplicou escalonamento ou aplicou de forma idêntica para ambos os modelos.	PARCIALMENTE. Aplicou o escalonamento no KNN, mas utilizou a função fit_transform nos dados de teste..	TOTALMENTE. Aplicou o StandardScaler apenas no KNN (fit_transform no treino/transform no teste) e justificou a ausência na Árvore.
12	Ajuste de Parâmetros e Combate ao Overfitting	NÃO REALIZADO. Não alterou os parâmetros K do KNN e max_depth da árvore.	PARCIALMENTE. Alterou os parâmetros, mas avaliou o desempenho apenas com os dados de treino.	TOTALMENTE. Testou ao menos 3 variações de parâmetros para cada modelo e apontou os pontos de sobreajuste no texto.
13	Avaliação da Acurácia e Veredito Final	NÃO REALIZADO. Não extraiu a acurácia dos modelos no teste ou omitiu a conclusão.	PARCIALMENTE. Calculou as acurácias no teste, mas não realizou a comparação dos valores ou omitiu a indicação do modelo final.	TOTALMENTE. Calculou a acurácia de ambos os modelos na base de teste, comparou os resultados numéricos e indicou o modelo de maior valor.

7 PLANO DE PROJETO

Ao construir a aplicação proposta, o estudante desenvolve as seguintes competências técnicas:

- Gerenciamento de repositórios de código utilizando Git e GitHub por meio do fluxo de ramificações e histórico de modificações.
- Execução de pipelines de dados em Python, mitigando ocorrências de vazamento de dados (*Data Leakage*) e assimetria estatística.
- Análise de desempenho de modelos de Inteligência Artificial por meio da comparação de valores numéricos de acurácia obtidos em conjuntos de dados independentes.

8 SOBRE O USO DE INTELIGÊNCIA ARTIFICIAL

É vedada a utilização de ferramentas de Inteligência Artificial (tais como ChatGPT, Claude, GitHub Copilot, entre outras) para a geração e a escrita integral do código deste projeto. O pipeline deve ser construído de forma autônoma pelo estudante.

É permitido o uso de ferramentas de IA exclusivamente para fins de consulta teórica, esclarecimento de dúvidas conceituais ou geração de exemplos genéricos de sintaxe. É proibida a delegação do desenvolvimento das funções ou blocos do projeto. Esta atividade tem como objetivo o contato direto com a escrita de código Python, a resolução de erros de execução e a fixação prática dos conceitos de modelagem.